UNIVERSIDAD AUTÓNOMA DEL CARIBE

MANUAL DE UTILIZACIÓN E IMPLEMENTACIÓN DEL SISTEMA Y SOFTWARE STIMO

Mateo David Castro Villegas
Duván Cohen Pérez
Carlos Gabriel Díaz Sáenz
Jean Pierre Coll Vásquez

ÍNDICE

- Introducción
- 1.1 ¿Qué es STIMO?
- 1.2 Arquitectura general del sistema
- Descripción general del código y su funcionamiento
- 2.1 Visión general: El núcleo del sistema terapéutico
- 2.2 Definiciones, pines y constantes globales
- 2.3 Estructuras y variables de estado
- 2.4 Configuración inicial en setup()
- 2.5 Estructura de tareas CORE0 y CORE1
- 2.6 Lógica WebSocket: Comunicación UI–ESP32
- 2.7 Control de temperatura (MAX6675 + TRIAC)
- 2.8 Control de electroterapia (PWM y modos TENS/EMS)
- 2.9 Monitor de flexión (MPU6050 dual)
- 2.10 Sistema de sesiones y pacientes
- 2.11 Envío de datos a Google Sheets
- 2.12 Interfaz web HTML optimizada
- Requerimientos del sistema
- 3.1 Hardware
- 3.2 Software
- Primeros pasos: Configuración y carga del firmware
- 4.1 Preparación del entorno
- 4.2 Carga del código al ESP32
- Modo de operación del STIMO
- 5.1 Flujo de uso del sistema
- 5.2 Control térmico
- 5.3 Electroterapia
- 5.4 Monitor de flexión
- 5.5 Guardado de sesiones

1. Introducción

1.1 ¿Qué es Stimo?

El firmware STIMO constituye el cerebro embebido de un sistema terapéutico integrado que combina termoterapia controlada mediante un TRIAC, electroestimulación TENS/EMS regulada con señal PWM, monitoreo del ángulo articular utilizando dos sensores MPU6050, registro automático de los datos clínicos en Google Sheets y una interfaz web interactiva que opera en tiempo real mediante WebSockets. Todo esto se ejecuta dentro de un ESP32, aprovechando su arquitectura de doble núcleo, donde cada core se encarga de funciones específicas para garantizar estabilidad, seguridad y capacidad de procesamiento simultáneo.

1.2 Arquitectura General del Sistema

STIMO es un sistema cliente–servidor embebido:

- El ESP32 ejecuta dos tareas:
  - CORE0 → Sensores, actuadores, electrotérmico, MPUs, PWM
  - CORE1 → WiFi, WebServer, WebSocket, envío a Google Sheets
- El ESP32 sirve una interfaz web interactiva donde el terapeuta controla:
  - temperatura
  - electroterapia
  - paciente/sesión
  - ángulo
  - guardado y registro
- Los datos se actualizan en tiempo real mediante WebSocket.

Esta arquitectura distribuida permite un sistema estable, responsivo y con baja latencia.

2. Descripción General del Código y su Funcionamiento

A continuación se describe el funcionamiento interno del firmware STIMO. Cada apartado incluye fragmentos del código original para facilitar la comprensión, exactamente como en el manual de Autotrigger.

2.1 Visión General: El Núcleo de STIMO

El firmware STIMO está escrito en C++ para el ESP32 y combina múltiples módulos: lectura de temperatura mediante MAX6675, control del TRIAC para termoterapia, generación de señales eléctricas para TENS/EMS mediante PWM, y medición de flexión articular con dos sensores MPU6050. Además, el ESP32 funciona simultáneamente como servidor web, manejando una interfaz en tiempo real mediante WebSockets.
En el código puede verse claramente la inclusión de todas las librerías necesarias para esta arquitectura:

```
#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <WebServer.h>
#include <WebSocketsServer.h>
#include <ArduinoJson.h>
#include <max6675.h>
#include <MPU6050.h>
#include <Wire.h>
```

Estas librerías habilitan:

- Conectividad WiFi del ESP32 mediante DHCP
- Comunicación cifrada HTTPS con Google Sheets
- Servidor web interno
- Canal WebSocket para comunicación en tiempo real
- Lectura del termopar MAX6675
- Lectura I2C de los dos MPU6050
- Serialización y envío de datos JSON hacia la interfaz web

La combinación de todos estos módulos hace que STIMO funcione como un sistema médico completo dentro del ESP32.

2.2 Definiciones, Pines y Constantes Globales

Toda la configuración física del sistema STIMO está centralizada dentro del namespace Config. Esta organización agrupa los pines y parámetros más importantes, igual que en el manual del Autotrigger, pero adaptado a los requisitos del sistema STIMO.

```
namespace Config {
  constexpr int TEMP_SCK_PIN = 18;
  constexpr int TEMP_CS_PIN  = 17;
  constexpr int TEMP_SO_PIN  = 19;
  constexpr int TRIAC_PIN    = 15;
  constexpr int ELECTRO_PIN  = 13;
  constexpr int MPU1_SDA     = 21;
  constexpr int MPU1_SCL     = 22;

  // PWM
  constexpr int PWM_RESOLUTION = 12;
  constexpr int PWM_FREQ       = 5000;

  // URL de Google Sheets
  const char* GAS_URL = "https://script.google.com/macros/s/AKfycbzfQ4mQS9g7I5Ukh

  // Wifi (usar DHCP)
  const char* WIFI_SSID     = "Redmi Note 14";      // ← Cambia por tu SSID
  const char* WIFI_PASSWORD = "Dcp17122002";   // ← Cambia por tu contraseña
}
```

1. Pines del sensor de temperatura MAX6675 (SPI)

Los pines del MAX6675 permiten leer la temperatura a través del termopar tipo K.

- TEMP_SCK_PIN = 18 → Señal de reloj SPI
- TEMP_CS_PIN = 17 → Chip Select
- TEMP_SO_PIN = 19 → Señal de salida del sensor

Estos pines deben estar conectados directamente al módulo MAX6675 del STIMO.

2. Control del TRIAC para termoterapia

- TRIAC_PIN = 15

Este pin activa el optoacoplador MOC que dispara el TRIAC.
Su funcionamiento es completamente digital:

- HIGH → Triac encendido → Calentamiento activo
- LOW → Triac apagado → Sin calor

3. Generación de pulsos eléctricos TENS/EMS

- ELECTRO_PIN = 13

Este pin utiliza el módulo PWM por hardware del ESP32:

- Frecuencia: PWM_FREQ = 5000 Hz
- Resolución: PWM_RESOLUTION = 12 bits

Esto permite ajustar la intensidad y forma del pulso eléctrico.

4. Comunicación con los módulos MPU6050

Ambos MPU comparten los mismos pines I2C del ESP32:

- MPU1_SDA = 21
- MPU1_SCL = 22

Cada MPU tiene una dirección distinta:

- MPU A → 0x68
- MPU B → 0x69

El sistema detecta automáticamente si cada uno está conectado y operativo.

5. Parámetros de red y Google Sheets

WIFI_SSID y WIFI_PASSWORD: conexión automática mediante DHCP
GAS_URL: script de Google Apps Script para almacenar las sesiones clínicas

2.3 Estructuras y Variables de Estado

El funcionamiento completo del STIMO depende de dos estructuras principales:

TherapySettings

```
struct TherapySettings {
  String mode = "TENS";
  int frequency = 50;
  int intensity = 0;
  int pulseWidth = 250;
  bool enabled = false;
};
```

Esta estructura contiene los parámetros de electroterapia:

- modo (TENS/EMS)
- frecuencia (2–100 Hz)
- intensidad (0–100 %, mapeado a PWM)
- ancho de pulso en microsegundos
- estado activo o desactivado

Controla completamente cómo se generan los impulsos a través del pin ELECTRO_PIN.

SessionData

```
struct SessionData {
  String patientName = "";
  String patientId = "";
  float tempSetpoint = 45.0;
  unsigned long tempStartTime = 0;
  unsigned long tempDuration = 0;
  TherapySettings therapySettings;
  unsigned long therapyStartTime = 0;
  unsigned long therapyDuration = 0;
  float maxAngle = 0;

  bool hasData() const {
    return patientName.length() > 0 && patientId.length() > 0;
  }

  void reset() {
    tempDuration = 0;
    therapyDuration = 0;
    maxAngle = 0;
    tempStartTime = 0;
    therapyStartTime = 0;
    therapySettings.enabled = false;
  }
};
```

Esta estructura almacena toda la información de una sesión:

- nombre y documento del paciente
- setpoint térmico
- tiempos de aplicación de calor
- parámetros de electroterapia
- duración de la terapia eléctrica
- ángulo máximo registrado mediante MPU6050

Además incluye dos funciones fundamentales:

hasData()

Evita que se envíen datos a Google Sheets sin un paciente registrado.

reset()

Reinicia la sesión sin borrar la identidad del paciente.

Variables de estado compartidas

```
volatile float currentTemp = 25.0;
volatile float currentAngle = 0.0;
volatile bool tempActive = false;
volatile bool mpu1_detected = false;
volatile bool mpu2_detected = false;
```

Estas variables son volátiles porque se actualizan en ambos Cores del ESP32, evitando inconsistencias entre tareas simultáneas.

2.4 Configuración Inicial en setup()

En STIMO, la función setup() no controla actuadores directamente. Su principal tarea es configurar la comunicación serial e inicializar las dos tareas paralelas del ESP32, asignándolas a los cores correspondientes.

```
void setup() {
  Serial.begin(115200);
  Serial.println("Iniciando Sistema Médico ESP32...");

  xTaskCreatePinnedToCore(
    Task0code,
    "Hardware",
    10000,
    NULL,
    2,
    &Task0,
    0
  );

  xTaskCreatePinnedToCore(
    Task1code,
    "Network",
    10000,
    NULL,
    1,
    &Task1,
    1
  );
}

void loop() {
  delay(1000);
}
```

1.Serial.begin(115200);

Activa el puerto serial a 115200 baudios. Esto permite imprimir información del sistema durante la depuración.

2.xTaskCreatePinnedToCore(... , 0)

Crea una tarea que se asigna al Core 0, llamada “Hardware”.

Core 0 ejecuta:

lectura del MAX6675

control de TRIAC

lectura de MPU6050

cálculo del ángulo

generación de pulsos TENS/EMS

3.xTaskCreatePinnedToCore(... , 1)

Crea la tarea “Network” en el Core 1, encargada de:

WiFi

servidor web

WebSocket

Google Sheets

4.El loop() queda vacío.

Esto permite que las tareas de FreeRTOS manejen el flujo completo sin interrupciones.

2.5 Estructura de Tareas CORE0 y CORE1

El ESP32 permite ejecutar código simultáneamente en dos núcleos. STIMO aprovecha esto para dividir responsabilidades.

2.5.1 Tarea CORE 0 – Task0code (Hardware)

```
void Task0code(void * pvParameters) {
  pinMode(Config::TRIAC_PIN, OUTPUT);
  digitalWrite(Config::TRIAC_PIN, LOW);

  setupPWM();

  Wire.begin(Config::MPU1_SDA, Config::MPU1_SCL);
  Wire.setClock(400000);
```

Explicación:

Core 0 configura:

- TRIAC como salida
- PWM para electroterapia
- Bus I2C a 400 kHz

Luego detecta los dos MPU6050:

```
  Wire.beginTransmission(0x69);
  if (Wire.endTransmission() == 0) {
    mpu2.initialize();
    if (mpu2.testConnection()) {
      mpu2_detected = true;
      mpu2.setDLPFMode(3);
    }
  }
```

El procedimiento se repite para la dirección 0x69.

Core 0 también ejecuta en bucle infinito:

- lectura del termopar cada 1 s
- lectura de ángulo cada 50 ms
- cálculo del ángulo diferencial
- filtrado exponencial
- actualización del ángulo máximo
- generación del pulso de electroterapia en tiempo real

2.5.2 Tarea CORE 1 – Task1code (Network)

Fragmento del código:

```
WiFi.mode(WIFI_STA);
WiFi.begin(Config::WIFI_SSID, Config::WIFI_PASSWORD);
```

Core1:

1. Se conecta a WiFi mediante DHCP.
2. Inicia el servidor web:

```
server.on("/", []() {
  server.send_P(200, "text/html", htmlPage);
});
```

3. Inicia el servidor WebSocket:

```
webSocket.begin();
webSocket.onEvent(webSocketEvent);
```

4. Envía datos a Google Sheets cada minuto.
5. Envía estado a los clientes WebSocket cada segundo.

2.6 Lógica WebSocket: Comunicación UI–ESP32

Toda la comunicación entre la interfaz web y el ESP32 se realiza en JSON.
Cuando el navegador envía un comando, el ESP32 lo interpreta en handleWebSocketMessage().

```
else if (action == "set_temp") {
  session.tempSetpoint = doc["value"];
}
```

Ejemplos de comandos que la página web envía:

- "register_patient"
- "toggle_temp"
- "toggle_therapy"
- "set_therapy_param"
- "save_session"
- "reset_angle"

Envío de estado desde ESP32 → UI

```
void sendStatus(uint8_t clientNum = 255) {
  DynamicJsonDocument doc(512);
  doc["temperature"] = currentTemp;
  doc["angle"] = currentAngle;
  doc["maxAngle"] = session.maxAngle;
  doc["setpoint"] = session.tempSetpoint;
  doc["tempActive"] = tempActive;
  doc["therapyActive"] = session.therapySettings.enabled;
  doc["mpu1Status"] = mpu1_detected;
  doc["mpu2Status"] = mpu2_detected;
  doc["ip"] = WiFi.localIP().toString();

  String json;
  serializeJson(doc, json);

  if (clientNum == 255) {
    webSocket.broadcastTXT(json);
  } else {
    webSocket.sendTXT(clientNum, json);
  }
}
```

La interfaz siempre refleja:

- temperatura actual
- ángulo actual
- ángulo máximo
- estado de TENS/EMS
- estado del TRIAC

2.7 Control de temperatura (MAX6675 + TRIAC)

La medición del MAX6675 se hace cada segundo.

```
// Temperatura (cada 1 s)
if (now - lastTempRead >= 1000) {
  lastTempRead = now;
  float rawTemp = thermocouple.readCelsius();
  if (!isnan(rawTemp) && rawTemp > 0 && rawTemp < 200) {
    currentTemp = currentTemp * 0.7 + rawTemp * 0.3;
  }
```

El filtro suaviza fluctuaciones del termopar.
Control de TRIAC:

```
    // Control simple ON/OFF
    if (tempActive && session.tempSetpoint > 0) {
      bool heaterOn = currentTemp < session.tempSetpoint;
      digitalWrite(Config::TRIAC_PIN, heaterOn ? HIGH : LOW);
    } else {
      digitalWrite(Config::TRIAC_PIN, LOW);
    }
  }
```

- Si está bajo el setpoint → TRIAC ON
- Si lo supera → TRIAC OFF

El sistema también detecta valores inválidos y los descarta.

2.8 Control de electroterapia (PWM y modos TENS/EMS)

STIMO genera pulsos eléctricos en tiempo real usando PWM de 12 bits.
Fragmento del código:

```
  ledcAttach(Config::ELECTRO_PIN, Config::PWM_FREQ, Config::PWM_RESOLUTION);
  ledcWrite(Config::ELECTRO_PIN, 0);
  Serial.println("PWM configurado para electroterapia.");
}
```

El pulso se crea así:

```
  if (pulseActive) {
    int pwmValue = map(session.therapySettings.intensity, 0, 100, 0, maxPwmValue);
    pwmValue = constrain(pwmValue, 0, maxPwmValue);
    ledcWrite(Config::ELECTRO_PIN, pwmValue);
  } else {
    ledcWrite(Config::ELECTRO_PIN, 0);
  }
}
```

Parámetros configurables desde la UI:

- mode: TENS o EMS
- freq: 2–100 Hz
- intensity: 0–100%
- pulseWidth: 50–500 µs

El firmware construye el estímulo eléctrico ajustando:

- periodo
- duty cycle
- tiempo de pulso HIGH

2.9 Monitor de flexión (MPU6050 dual)

Se leen ambos sensores:

```
mpu1.getMotion6(&ax1, &ay1, &az1, &gx1, &gy1, &gz1);
mpu2.getMotion6(&ax2, &ay2, &az2, &gx2, &gy2, &gz2);
```

Luego se calcula el ángulo:

```
float angle1 = atan2(acc1_y, sqrt(acc1_x * acc1_x + acc1_z * acc1_z)) * 180.0 / PI;
float angle2 = atan2(acc2_y, sqrt(acc2_x * acc2_x + acc2_z * acc2_z)) * 180.0 / PI;

float angleDiff = abs(angle1 - angle2);
```

El sistema aplica filtrado:

```
float smoothedAngle = lastAngle * 0.85 + angleDiff * 0.15;
```

Finalmente:

- currentAngle se actualiza
- session.maxAngle se actualiza automáticamente

2.10 Sistema de sesiones y pacientes

El terapeuta registra:

- nombre del paciente
- documento

Cuando esto sucede:

```
DynamicJsonDocument response(256);
response["patientRegistered"] = true;
response["patientName"] = session.patientName;
response["patientId"]   = session.patientId;
```

Cada sesión mide:

- duración de calor
- duración de electroterapia
- ángulo máximo
- parámetros eléctricos

El comando "new_session" reinicia la sesión.

2.10 Sistema de sesiones y pacientes

El terapeuta registra:

- nombre del paciente
- documento

Cuando esto sucede:

```
DynamicJsonDocument response(256);
response["patientRegistered"] = true;
response["patientName"] = session.patientName;
response["patientId"]   = session.patientId;
```

Cada sesión mide:

- duración de calor
- duración de electroterapia
- ángulo máximo
- parámetros eléctricos

El comando "new_session" reinicia la sesión.

2.11 Envío de datos a Google Sheets

El ESP32 hace una petición HTTPS:

```
client.println("POST " + String(Config::GAS_URL) + " HTTP/1.1");
client.println("Host: script.google.com");
client.println("Content-Type: application/json");
client.println("Content-Length: " + String(json.length()));
client.println("Connection: close");
client.println();
client.println(json);
```

Los datos enviados incluyen:

- nombre
- ID
- setpoint térmico
- duración de calor
- modo de terapia
- intensidad
- frecuencia
- ancho de pulso
- duración total
- ángulo máximo

Si Google responde "success", la sesión se registra.

2.12 Interfaz Web HTML optimizada

El ESP32 sirve un archivo HTML completo almacenado en PROGMEM:

```
server.on("/", []() {
  server.send_P(200, "text/html", htmlPage);
});
```

La página incluye:

- modal de registro
- panel de temperatura
- panel de electroterapia
- panel de ángulo
- slider dinámico
- UI estilo “green cyber”
- indicadores de conexión
- actualizaciones WebSocket en tiempo real

El diseño está optimizado para pantallas móviles y tablets clínicas.

3. Requerimientos del Sistema

Esta sección describe los elementos necesarios para garantizar el funcionamiento correcto del firmware STIMO. Se divide en requerimientos de hardware y de software, tal como se presenta en manuales técnicos de sistemas embebidos.

3.1 Hardware

Para ejecutar correctamente el firmware STIMO y garantizar que cada módulo funcione como se diseñó, se requiere el siguiente hardware:

1.Microcontrolador ESP32

Es el cerebro del sistema. Debe ser un modelo con doble núcleo (cualquier DevKit estándar). Su función es manejar simultáneamente las tareas de hardware y red mediante FreeRTOS.

2.Sensor de temperatura MAX6675 con termopar tipo K

Este módulo lee la temperatura del sistema y envía valores digitales al ESP32 mediante comunicación SPI.

3.TRIAC y optoacoplador MOC

Permiten controlar el elemento calefactor del sistema mediante señales digitales. El TRIAC es activado y desactivado por el pin configurado en el firmware.

4.Electrodo de electroterapia y circuito de salida TENS/EMS

El sistema usa un canal PWM del ESP32 para generar pulsos eléctricos controlados. Este módulo debe estar correctamente aislado para seguridad del paciente.

5.Dos módulos MPU6050

Se usan para medir el ángulo articular del paciente mediante acelerometría diferencial. Ambos comparten el bus I2C del ESP32 con direcciones distintas (0x68 y 0x69).

6.Fuente de alimentación estable

Se recomienda separar la alimentación del sistema de potencia (calefacción y electroterapia) del ESP32 para evitar ruidos eléctricos y reinicios inesperados.

3.2 Software

Para compilar, cargar y ejecutar el firmware STIMO, se requiere el siguiente software:

1.Arduino IDE

Es el entorno principal donde se compila el código y se sube al ESP32.

2.Librerías necesarias

Las siguientes librerías deben estar instaladas:

- WiFi.h
- WebServer.h
- WebSocketsServer.h
- WiFiClientSecure.h
- ArduinoJson.h
- max6675.h
- MPU6050.h
- Wire.h

3.Gestor de tarjetas ESP32

Debe estar instalado desde el Gestor de Placas del Arduino IDE. Se usa para que el IDE reconozca las características del ESP32 y pueda compilar el proyecto correctamente.

4.Google Apps Script

Un pequeño script se utiliza para recibir los datos enviados por STIMO y almacenarlos en una hoja de Google Sheets. El enlace a este script se incluye en el firmware.

4. Primeros Pasos: Configuración y Carga del Firmware

Esta sección explica paso a paso cómo preparar el entorno de desarrollo y cargar el código al ESP32. Es equivalente a las instrucciones vistas en el manual de Autotrigger, pero adaptadas a STIMO.

4.1 Preparación del Entorno

Antes de cargar el firmware STIMO, deben seguirse los siguientes pasos:

1. Abrir el Arduino IDE.
2. Ir a Archivo > Preferencias.
3. En “URLs Adicionales de Gestor de Tarjetas”, añadir la siguiente URL: https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
4. Ir a Herramientas > Placa > Gestor de Tarjetas.
5. Buscar “ESP32” e instalar la tarjeta oficial de Espressif.
6. Ir a Programa > Incluir Librería > Administrar Bibliotecas.
7. Instalar las librerías:
   - ArduinoJson
   - WebSockets
   - MAX6675
   - MPU6050
8. Verificar que el computador detecta el puerto USB del ESP32.

4.2 Carga del Código al ESP32

1. Abrir el archivo del firmware STIMO en el Arduino IDE.
2. Conectar el ESP32 al computador mediante cable USB.
3. Ir a Herramientas > Placa y seleccionar: ESP32 Dev Module
4. Ir a Herramientas > Puerto y seleccionar el puerto COM detectado.
5. Presionar el botón “Subir” en el Arduino IDE.
6. Esperar a que el código compile y se transfiera al ESP32.
7. Abrir el Monitor Serial para verificar que el sistema inicia correctamente y se conecta a la red WiFi.

En este punto el sistema queda listo para ser utilizado.

5. Modo de Operación del STIMO

Esta sección describe el comportamiento completo del sistema STIMO durante una sesión real, incluyendo el flujo de trabajo y la función de cada módulo del firmware. Esta parte del manual está diseñada pensando en terapeutas, evaluadores y desarrolladores técnicos.

5.1 Flujo de Uso del Sistema

El flujo general del sistema STIMO sigue el siguiente orden:

1. El ESP32 arranca e intenta conectarse a la red WiFi configurada.
2. El sistema inicia el servidor web y WebSocket.
3. El terapeuta accede a la interfaz escribiendo la IP del dispositivo en un navegador.
4. La página solicita registrar el nombre y documento del paciente.
5. Una vez registrada la información, se desbloquean los módulos de:
   - control térmico,
   - electroterapia,
   - monitor de flexión.
6. El terapeuta ajusta el setpoint de temperatura o los parámetros TENS/EMS.
7. El sistema registra automáticamente:
   - temperatura actual,
   - ángulo de flexión,
   - duración de cada modalidad de terapia,
   - parámetros utilizados.
8. Al finalizar, el terapeuta presiona “Guardar sesión”.
9. El ESP32 envía los datos a Google Sheets y confirma si el registro fue exitoso.

5.2 Control Térmico

El módulo térmico utiliza un sensor MAX6675 conectado mediante SPI y controla un TRIAC para activar o desactivar la resistencia térmica.

El uso es el siguiente:

1. El usuario ajusta el setpoint desde la interfaz.
2. El ESP32 compara la temperatura medida con el setpoint.
3. Si la temperatura está por debajo, activa el TRIAC.
4. Si está por encima del límite, lo apaga.
5. El sistema registra el tiempo total que la terapia térmica estuvo activa.
6. Si el usuario desactiva manualmente la terapia, STIMO apaga inmediatamente la salida térmica.

El sistema incluye un filtro para suavizar variaciones rápidas y proteger el TRIAC.

5.3 Electroterapia

El control TENS/EMS del STIMO funciona mediante PWM de alta resolución. Desde la interfaz se pueden ajustar:

- frecuencia del pulso,
- intensidad del pulso,
- ancho de pulso,
- modo (TENS o EMS),
- activación o desactivación.

Al iniciar la terapia eléctrica:

1. El firmware construye pulsos eléctricos periódicos.
2. La intensidad se convierte a un valor PWM entre 0 y 4095.
3. El periodo se ajusta según la frecuencia seleccionada.
4. El pulso se activa durante el tiempo definido en microsegundos.

La duración de la terapia eléctrica se mide automáticamente.

5.4 Monitor de Flexión

El sistema mide el ángulo del paciente usando dos MPU6050 colocados uno sobre el muslo y otro sobre la pierna. El código:

1. Lee las aceleraciones de ambos sensores.
2. Calcula la inclinación en grados de cada uno.
3. Obtiene el ángulo diferencial entre ambos segmentos.
4. Filtra la señal para evitar ruido.
5. Actualiza continuamente el ángulo máximo logrado.

Este valor es útil para evaluar la evolución del paciente durante la rehabilitación.

5.5 Guardado de Sesiones

Cuando el terapeuta finaliza la sesión:

1. Presiona el botón “Guardar” desde la interfaz.
2. El ESP32 empaqueta todos los datos en formato JSON:
    - nombre y documento del paciente,
    - duración de terapia térmica,
    - duración de electroterapia,
    - parámetros TENS/EMS,
    - ángulo máximo alcanzado.
3. El sistema realiza una petición HTTPS hacia Google Apps Script.
4. Si la respuesta contiene la palabra “success”, el sistema confirma que los datos se almacenaron correctamente.
5. La interfaz muestra una notificación visual indicando éxito o error.

Este registro permite mantener control clínico de cada paciente y facilita análisis posteriores.

UNIVERSIDAD AUTÓNOMA DEL CARIBE

MANUAL DE UTILIZACIÓN E IMPLEMENTACIÓN DEL SISTEMA Y SOFTWARE STIMO

Mateo David Castro Villegas
Duván Cohen Pérez
Carlos Gabriel Díaz Sáenz
Jean Pierre Coll Vásquez